

Multi-Agent Knowledge Sharing with RAG and MCP

Modern AI “agents” (LLM-based assistants) can be orchestrated to collaborate on software development tasks by sharing context through a retrieval-augmented generation (RAG) pipeline and a Model Context Protocol (MCP) server. In this architecture, **RAG** supplies relevant project data (code, docs, logs) to each agent on demand, while an **MCP-based memory service** lets agents log and retrieve facts across sessions. For example, GitHub’s Copilot uses a vector-backed index of the codebase and “smart search” to find semantically similar code files for each query ¹. Meanwhile, agents write important discoveries (like “API version must match in these files”) into a centralized memory store via a tool interface ². An MCP server makes this memory store and other tools accessible to any agent, **standardizing** how models fetch and update context ³ ⁴.

Architecture and Data Stores

A typical design involves: (1) a **central knowledge store** (e.g. a vector database or document index) holding project artifacts and memories, (2) **agents** (LLMs or chains of LLMs) performing tasks, and (3) an **MCP server** exposing search/memory tools. As the Microsoft multi-agent reference architecture suggests, agents are coordinated by an orchestrator that preserves context and routes tasks, while knowledge bases and vector stores hold contextual data and memories ⁵. In practice, the system might index the entire codebase (or documentation) with embeddings (e.g. using OpenAI or open-source embedding models) and store them in a scalable vector DB (Pinecone, Qdrant, Weaviate, Milvus, FAISS, etc. are common choices ⁶). These allow semantic search (“what code relates to this bug?”) rather than just keyword matching.

Agents query this store via RAG: they embed the current prompt (bug description, code snippet, etc.), retrieve top-k similar chunks, and include those in the LLM prompt. In GitHub Copilot Chat, for example, the service “performs a semantic search on the [code] index” using advanced embeddings, finds the files most similar to the user’s request, and adds them into the prompt ¹. Frameworks like LangChain or LlamaIndex can orchestrate this retrieval in Python: they handle chunking the data, embedding queries, and calling the vector DB for results. The retrieved code snippets or docs then ground the LLM’s output.

Alongside RAG, we maintain a **long-term memory store** for non-textual “facts” and lessons learned. This is often implemented as a specialized MCP server with tools like `store_memory(key,value)` and `search_memory(query)`. When an agent discovers something important (e.g. a design rule, a bug fix, or a standard), it calls the memory tool, sending structured info (subject, fact, citations) to the store. For instance, Copilot’s code-review agent might invoke a memory tool to save:

```
{
  "subject": "API version synchronization",
  "fact": "API version must match between client SDK, server routes, and documentation.",
  "citations": ["src/client/sdk/constants.ts:12", "server/routes/api.go:8", "docs/api-reference.md:37"],
```

```

    "reason": "If the API version is not kept synchronized, integrations can
fail."
}

```

This entry is stored (e.g. in a vector DB or database) so that any other agent can retrieve and use it ². The diagram below sketches this flow: an agent output triggers a call to the memory service (via MCP), which updates the persistent memory store.

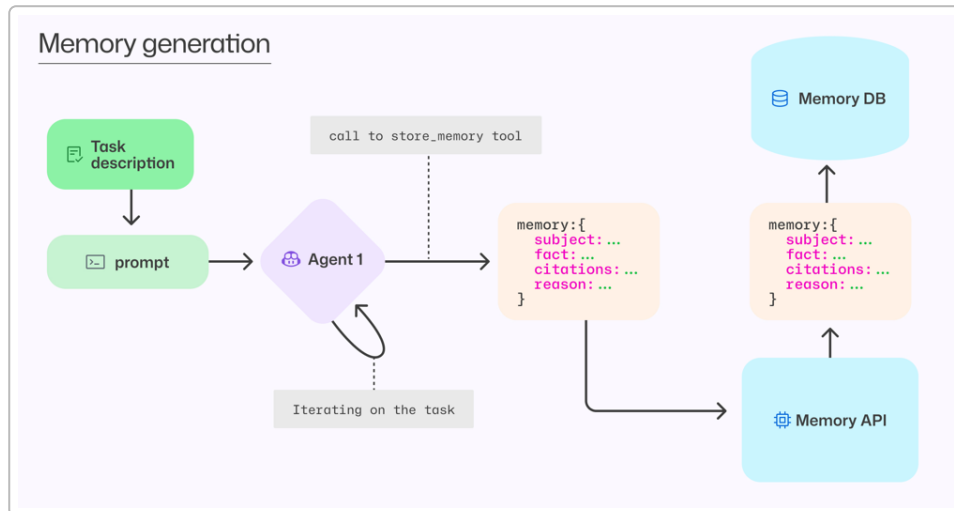


Illustration: Agent writes a memory entry via an MCP-based tool. The memory (subject/fact with context) is saved centrally. Future agents can retrieve it to enrich prompts or guide decisions ² ⁷.

Whenever a new agent session starts, the system can fetch the most relevant memories for that codebase and prepend them to the prompt ⁷. For example, GitHub retrieves “the most recent memories for the target repository” and includes them automatically ⁷. Agents also run verification steps: they re-check the cited code to ensure the fact is still valid, updating or discarding memory entries if the code has changed ⁸ ⁷. In effect, each agent contributes to a shared knowledge base about the project, and each session is enriched by past learnings.

Technology Stack

In Python, a feasible stack includes:

- **LLM Frameworks/Agents:** OpenAI’s Agents SDK (supports MCP) or other agent libraries (LangChain Agents, AutoGPT-style systems, Semantic Kernel, etc.) allow you to define tools and pipelines. The SDK’s MCP integration (e.g. `MCPServerStreamableHttp`) can connect your memory/search server to any LLM client ³.
- **Vector/Document Store:** A managed or self-hosted vector DB (Pinecone, Weaviate, Qdrant, Milvus, etc.) to index text, code, docs, and possibly chat logs. Alternatively, a full-text DB (SQLite FTS5, Elasticsearch) can store conversation transcripts. As one developer notes, simply writing chat logs to SQLite with FTS search lets an agent keyword-search past sessions on demand ⁹ ¹⁰. (In many coding contexts, keyword recall suffices because the agent can craft targeted queries ¹¹ ¹².) But for broad semantic search (e.g. “show me where we discussed database migrations”), embeddings+vector search is the standard approach.
- **Context/Memory Storage:** The same or a separate datastore can hold structured memory entries. Many designs use the vector DB for these too (by embedding facts), or a key-value store

(Redis/SQL) for straightforward lookups. GitHub's memory (e.g. the JSON example above) could be stored as a document in a vector DB or a JSON field in a SQL DB; what matters is efficient search over the "subject" or content.

- **API/MCP Server:** A Python web service (e.g. FastAPI) implementing the MCP spec can expose tools like `search_docs(query)` and `store_memory(memory_obj)`. The [Model Context Protocol](#) provides a schema for such tools ³ ⁴. The OpenAI Agents Python SDK even includes helpers (`FastMCP`) to auto-generate tool definitions from Python functions ¹³. Your LLM clients (e.g. Copilot-like code agents, ChatGPT-based bots, etc.) simply need to call these tools via HTTP or stdio according to MCP. This decouples them from the backend: you can swap out LLM models or memory implementations without changing prompts.
- **Integration Tools:** For code context, use source-control data. Many systems index Git repos: GitHub Copilot, for instance, builds an index of the repo (or lets you push code to a private RAG index) ¹⁴. You could similarly fetch the latest code or docs, chunk them (e.g. by file, class, or function), and feed them into the vector store.

Example Workflow

1. **Setup & Indexing:** Onboarding a project, the system scans the codebase and documentation, converts each logical chunk to embeddings (using an embedding model), and stores them in the vector DB. It also creates a memory table scoped to this project.
2. **Agent Task & Retrieval:** A developer asks a question or issues a command (e.g. "Why is this bug happening?"). The front-end agent formulates a prompt describing the bug and any available code snippet. It calls a `search_docs()` tool (via MCP) with the user's query. The MCP server queries the vector DB for semantically similar code/comments and returns the top passages. The prompt is augmented with these.
3. **LLM Reasoning:** The (first) LLM agent uses the enriched prompt to suggest actions or answers. It might, for example, identify a missing check in the code or a misunderstanding of the design spec. It can also decide to take an action, like "let me try writing a patch to fix X."
4. **Memory Logging:** If the agent uncovers a generalized insight (e.g. "We found that configuration Y must not be null"), it calls `store_memory()` with a structured entry (subject/fact, with references). This entry goes into the central memory store ². For instance, if debugging revealed a standard command to run, the agent could store: "Fact: Run `npm install` before building; Citation: Jenkinsfile line 23; Reason: build fails without dependencies."
5. **Handoff to Next Agent:** If a coding agent wrote or tested a patch, a *different* agent (e.g. a code reviewer or CI helper) may run next. At the start of its session, it automatically fetches the latest memory entries for the repo and sees the new fact. If it notices code is being changed, it uses that memory – for example, it flags if someone updated one file's version number without updating the others, as GitHub's Copilot did in their example ² ¹⁵.
6. **On-Demand Recall:** During a session, an agent can also explicitly query past conversations or memories. For instance, a "chat" agent might have a tool `search_conversations(keyword)` (backed by the FTS index of past chats ⁹). It can say, "Recall any mention of 'database migration'," effectively letting it retrieve relevant logs of previous sessions.

Over time, the memory fills with project-specific facts (coding standards, discovered bugs, architectural notes). When any agent starts a new task, these facts are fed in. GitHub reported that including such memories measurably improved Copilot's performance (e.g. 7% higher PR merge rates when memory was on) ¹⁶. In our system, the LLM uses MCP to "discover" these memories as available tools, ensuring that no agent has to manually repeat context setup ¹⁷ ⁵.

Implementation Plan

1. **Proof-of-Concept:** Start with a single LLM (e.g. GPT-4 or Claude via API) in Python. Build a small Flask/FastAPI service with two endpoints: `POST /mcp/search` (reads query, returns top-RAG results) and `POST /mcp/memory` (accepts memory objects to store). Internally use a simple vector store (e.g. SQLite+FAISS or an in-memory Pinecone test index). Write a Python script that demonstrates the loop: agent asks a question, service returns context, and agent “adds” a memory.
2. **MCP Integration:** Incorporate the OpenAI/Anthropic Agents SDK so the LLM sees these as MCP tools. Define the `search_docs` and `remember` tools via type hints (or JSON schema) so the SDK can call them. Test calling the agent programmatically: it should call the tools and get results.
3. **Multi-LLM Support:** Extend the Python system so that any LLM (OpenAI, Anthropic, local LLaMA, etc.) can connect. If needed, run a minimal MCP server process per provider (using `MCPServerStreamableHttp` from OpenAI’s SDK). Each agent’s config simply points to the same memory/RAG endpoints. Thus, Copilot-like agents and GPT-based agents could both call `search_docs()` on the same backend.
4. **Data Collection:** Implement scraping of code and logs. For code, use `git grep` or GitHub’s API to pull files. For chat logs (from a team chat or VSCode ChatGPT plugin), capture transcripts. Convert all textual artifacts to vectors and index them.
5. **Memory Schema and Curation:** Decide on memory formats (possibly JSON with fields subject/fact/citations). Build simple client-facing UI or skill for a human to approve agent-stored memories. For example, on each `store_memory` call, have the agent confirm and perhaps provide an “Are you sure?” prompt, storing only after approval. Optionally add background curation: a nightly job could remove duplicates, expired branches (as GitHub considered), or conflicting memories.
6. **Testing & Iteration:** Simulate real workflows. For a given repository, run agents on existing issues/PRs and let them populate memory. Then start new issues and observe whether past memory helps. Use unit tests or user studies to measure improvements (e.g. correct bug fixes, speed).

Challenges and Considerations

- **Staleness and Validity:** Codebases change. A fact true today may become false after a refactoring. As GitHub notes, “the core challenge...is ensuring that any stored knowledge remains valid as code evolves across branches and time” ⁸. To mitigate this, each memory entry should include precise citations (file paths/line numbers). Agents must verify citations before trust – if the code has moved or changed, they should update or drop the memory ⁸ ⁷. Periodic cleanup or version-tagging of memory can help expire outdated info.
- **Access Control:** In team settings, not all agents/users should see all memories. Design the store with scoping: e.g. partition memories by repository or project, and enforce read/write permissions. GitHub’s system restricts memories to users with repo access ¹⁸. Initially you may allow all context internal access, but for wider deployment, integrate with your auth (e.g. only allow memories to be read by agents acting for authorized users).
- **Index Size and Performance:** A large project can produce thousands of memory entries and code chunks. Efficient vector search requires pruning or weighting. You may implement relevance ranking (e.g. only fetch “recent” or high-scoring memories for each session, as GitHub suggests) ⁷. Caching common queries or using fast in-memory DBs (Redis, SQLite FTS) can reduce latency.

- **Multi-Agent Coordination:** If two agents act in parallel, they might clash. For example, one agent's memory write might race with another's read. Use transactions or simple locking on the memory store. Also, define ownership: maybe a "conductor" agent schedules tasks and aggregate memory contributions to avoid duplication.
- **System Complexity:** Integrating many LLMs and tools can become complex ("prompt spaghetti" as one dev put it). MCP helps by centralizing tool definitions ¹⁷, but careful design is needed so that agents know which tools to call and when. Rigorous testing of the agent workflows (possibly with simulated inputs) is essential.
- **Data Quality:** Agents can hallucinate or learn incorrect rules. Build in safeguards: each memory tool invocation should require the agent to cite actual evidence. GitHub's pipeline has the agent double-check the code at the cited locations ² ⁷. You might enforce that only tools (like a code-read tool) can feed memory, rather than free-text memory injection by the model alone.

In summary, implementing RAG+MCP for code-oriented multi-agent AI involves setting up a central vector/knowledge store, building an MCP server for search and memory tools, and integrating Python-based LLM agent frameworks. By following the examples of systems like GitHub Copilot — which create structured "memories" and use semantic search over the codebase ² ¹ — you can enable different coding agents to share context seamlessly. The key is careful design of the data store, strict validation of stored knowledge, and a robust orchestration layer that feeds the right context to each agent. Properly done, this can dramatically reduce repetitive context-giving and let agents build on each other's work.

Sources: We drew on the latest MCP/RAG resources and real-world examples, including Anthropic's MCP announcement ⁴, OpenAI's Agent SDK docs ³, GitHub's Copilot memory engineering blog ² ⁷, Microsoft's Copilot Chat context docs ¹, a multi-agent architecture guide ⁵, and community posts on LLM memory design ⁶ ⁹. Each informs parts of the proposed approach.

¹ ¹⁴ How Copilot Chat uses context - Visual Studio (Windows) | Microsoft Learn

<https://learn.microsoft.com/en-us/visualstudio/ide/copilot-context-overview?view=visualstudio>

² ⁷ ⁸ ¹⁵ ¹⁶ ¹⁸ Building an agentic memory system for GitHub Copilot - The GitHub Blog

<https://github.blog/ai-and-ml/github-copilot/building-an-agentic-memory-system-for-github-copilot/>

³ Model context protocol (MCP) - OpenAI Agents SDK

<https://openai.github.io/openai-agents-python/mcp/>

⁴ Introducing the Model Context Protocol \ Anthropic

<https://www.anthropic.com/news/model-context-protocol>

⁵ Reference architecture - Multi-agent Reference Architecture

<https://microsoft.github.io/multi-agent-reference-architecture/docs/reference-architecture/Reference-Architecture.html>

⁶ ¹⁷ LLM, RAG, AGENT And MCP - DEV Community

<https://dev.to/tak089/llm-rag-agent-and-mcp-cgo>

⁹ ¹⁰ ¹¹ ¹² What I Learned Building a Memory System for My Coding Agent : r/ClaudeCode

https://www.reddit.com/r/ClaudeCode/comments/1r1w397/what_i_learned_building_a_memory_system_for_my/

¹³ Build an MCP server - Model Context Protocol

<https://modelcontextprotocol.io/docs/develop/build-server>